Implementation and Experiment Backlog

ACIIDS 2027 paper: *Privacy-Preserving, Publicly Verifiable EUDR Compliance over EPCIS Event Data with Recursive Zero-Knowledge Proofs*

Repository: https://github.com/vanthinh26102005/nckh-ise-blockchain

Created 4 October 2026 · Submission deadline 31 October 2026 (EasyChair)

How to use this list

Every red slot `[ID: ...]` in `aciids2027/main.tex` is produced by the `\pending{ID}{...}` macro. Each ID below states which slot it fills, the exact command, the acceptance criterion and the output path. A slot may be filled only with numbers that come from committed metadata and raw output stored outside Git, as required by the repository's rules (`results/README.md`). Every new number must also be added to `aciids2027/evidence_ledger.md`. Before submission, set `\finaltrue` in `main.tex`; compilation then fails while any slot remains.

Commands assume the repository root as the working directory. Raw output goes under `/datastore/uitchain/...` (or any path outside Git).

Critical path. `T-SEC-01` → `T-E2-01` → `T-E3-01` (gate) → `T-E1-03` / `T-E2-02` / `T-E3-02` → `T-E4-01` → `T-E4-02` → `T-ABS-01`. Security fixes come first, because they change the leaf and epoch ABI and therefore invalidate every proof measured earlier.

1 Security fixes required by the paper's analysis (Section 5)

`T-SEC-01` Registry binding of R_C and c_G (condition B1) — fills: `B1 text, Prop. 1`
hardware: dev laptop · est. wall-clock: 2 days

- **Problem.** `epoch.rs::evaluate` does not check, and `EpochAggregateAnchor.sol` does not see, the leaf fields `certificate_root` and `polygon_commitment`. A prover can therefore use its own credential tree or polygon.
- **Change.**
  - In `crates/sp1-e2e/policy/src/epoch.rs`, require every child to carry the same `certificate_root` and `polygon_commitment`.
  - Append both fields to `EpochPublicValues`, which changes the ABI from 288 to 352 bytes.
  - Hash both fields into Γ.
  - In `EpochAggregateAnchor.sol` and `EpochAnchor.sol`, add a `registry` (owner-settable mapping of accepted roots) and revert if either value is not registered.
- **Tests.**
  - Rust unit test: an epoch with mismatching roots is rejected.
  - Anvil smoke (`contracts/test/anvil_e3_smoke.mjs`): an unregistered root reverts.
  - Rerun `make e3-check`.
- **Paper.** Change B1 from "not yet enforced" to "enforced". Update the 288-byte figure in Section 4.2 and the contract description in Section 4.3.

`T-SEC-03` Key-bound nullifier and wider index (condition B2) — fills: `B2 text`
hardware: dev laptop · est. wall-clock: 1.5 days

- **Change.**
  - Derive $s = \mathrm{PRF}_k(\ell)$, where k is the producer's nullifier key.
  - Commit $\mathsf{H}(k)$ inside the C2 credential leaf (`certificate_leaf`).
  - The guest proves knowledge of k and recomputes s, so it can no longer accept an arbitrary `nullifier_secret`.
  - Replace the 32-bit index with the full 256-bit digest. Either use a 256-level sparse Merkle tree with cached empty-subtree hashes (`SparseNullifierMap` already has the needed structure) or an indexed Merkle tree.

- **Tests.** Claiming the same lot with two secrets must fail, and the existing tamper and replay tests must still pass.
- **Paper.** Update C5 in Definition 1 and the leaf description in Section 4.1, and mark B2 as enforced.

`T-SEC-02` Hiding (salted) commitments (condition B3) — fills: `B3 text`
hardware: dev laptop · est. wall-clock: 1 day

- **Change.**
  - Add 32-byte nonces r_G and r_D to `PolicyInput`.
  - Set $c_G = \mathsf{H}(\text{tag}_G, r_G, k, G)$.
  - Build the event commitment as a Merkle root over salted event leaves $\mathsf{H}(\text{tag}_E, r_i, \mathsf{bytes}(e_i))$. This also enables subset openings for `T-SD-01`.
- **Paper.** Mark B3 as enforced and describe D as a Merkle root in Section 3.4.

2 Data model and functionality

`T-DM-01` EPCIS 2.0 JSON-LD → `EpcisEventV1` mapper — fills: `Sec. 3.2 slot`
hardware: dev laptop · est. wall-clock: 3 days

- Implement the mapping in Table 2 of the paper as `crates/sp1-e2e/policy/src/epcis_jsonld.rs`, or as a Python tool in `scripts/`:
  - `eventID` → 64-bit truncated SHA-256;
  - LGTIN → lot;
  - `readPoint` geo URI → micro-degrees;
  - `certificationInfo` → κ;
  - `bizStep` → role code.
- Validate against GS1 EPCIS 2.0 example documents. Reject events without geolocation.
- Aggregation and transformation events: encode parent and child lot edges, and add a continuity check that every input lot of a transformation has an anchored nullifier. This is a stretch goal. If it is not done by 25 Oct, keep it as future work in Section 7.

`T-SD-01` Selective-disclosure view prototype — fills: `Sec. 4.5 slot`
hardware: dev laptop · est. wall-clock: 2 days

- Depends on `T-SEC-02`.
- CLI `view-open` with options `-lot` and `-fields polygon,events[0..3],credential`. It outputs a JSON opening bundle.
- CLI `view-verify` recomputes c_G, the event Merkle root, Γ and R_C, and checks them against the anchored event log on Anvil.
- Measure, for three views (regulator, importer, certifier): bundle size in bytes and verification time in ms, 30 repetitions each.
- Report in one sentence or a small table in Section 4.5.

`T-C1-01` Plot-polygon and multi-polygon geofence (reviewer W4; future work if late) — fills: `Sec. 7 slot`
hardware: dev laptop · est. wall-clock: 3 days

- EUDR Art. 2(28) describes plots larger than 4 ha as polygons. C1 should therefore test *polygon disjointness* (plot polygon ∩ forest-loss polygon = ∅) using exact integer segment-intersection tests, and not only point exclusion.
- C1 over a registry tree of forest-loss polygons per map tile. The prover shows that the point lies outside every polygon of its tile, together with a tile-completeness proof (a Merkle path of the tile's polygon list against the registered root).
- Source polygons: forest-loss alerts after 2020, e.g. Hansen/GFW, rasterised into vector polygons for the Central Highlands.
- If this is not done, keep the sentence in Section 7 and delete the slot.

3 Experiments (Section 6)

`T-E2-01` 3-Raft Fabric topology and host record — fills: `Sec. 6.2 slot`
hardware: A100 host (same host for Fabric, Anvil and prover) · est. wall-clock: 0.5 day

- Bring up the topology and check it:
  ```
  bash infra/fabric/raft/bootstrap.sh up
  python3 scripts/e4_preflight.py ...  # must list 3 orderers + 2 peers
  ```
- Record in the paper: CPU model, cores, RAM, GPU model and driver, CUDA, OS, Fabric image tags, Anvil version and SP1 version. Take them from `preflight.json`.

T-E3-01 Recursive epoch gate (aggregate proof → Anvil receipt) fills: `Table 5,` G_a`; Sec. 6.6`
hardware: A100 · est. wall-clock: 0.5 day

- Run the gate:
  ```
  make e3-gate E3_FIXTURE=/datastore/uitchain/e3/gate.json
  E3_RECEIPT=/datastore/uitchain/e3/receipt.json
  ```
- **Acceptance.**
  – `results/e3/gate_manifest.json` exists, with receipt `status=1` and the expected new root.
  – The tampered aggregate is rejected.
- Record the aggregate gas G_a and the calldata bytes. They give the ratio mG_v/G_a used in Section 6.6.

T-E1-01 Path B rerun at production FRI parameters fills: `Table 4 row "C1-C5 at ≥100-bit"`
hardware: Mac (8-core) and A100 host · est. wall-clock: 1 day

- In `crates/e1-bench/src/templates.rs`, set `FRI_LOG_BLOWUP=3`, `FRI_NUM_QUERIES=33` and `query_proof_of_work_bits=16`. That gives $\approx 3 \cdot 33 + 16 = 115$ conjectured bits. Add a unit test that asserts at least 100 bits.
- Run:
  ```
  make e1-strict
  ```
- Use events $\in \{8, 16, 32, 64\}$, 30 seeds and *all* of C1–C5, including C1. Check that C5 is the current depth-32 update and not the empty-leaf MVP.
- Align the Path B parameters with Path A, as asked by reviewer W5: credential depth (run both 6 and 16), and document the hash difference (Poseidon2 versus SHA-256) and C4 (preimage versus Ed25519).
- Replace the archived rows of Table 4, or keep them as a "low-security" column if space allows. Update the C2/C3 text in Section 6.4.

T-E1-02 Path B recursive wrapper benchmark fills: `Table 4 row "Recursive wrapper"`
hardware: A100 host CPU · est. wall-clock: 1 day

- Run:
  ```
  cargo run -release -p e1-bench -features recursion - -circuits wrapper_recursive_plonky3 -events
  8,64 -seeds 10 -out <raw>
  ```
- Report prove time, verify time, proof size (postcard bytes) and peak RSS.

T-E1-03 SP1 leaf cost, CPU vs. GPU A/B fills: `Table 4 row "SP1 leaf"`
hardware: A100 · est. wall-clock: 1 day

- Reuse the CPU/GPU A/B from `make e4-preflight`: leaf proofs at $n = 8$ and $n = 64$, plus the two-lot aggregate.
- Report compressed-proof time, Groth16-wrap time, proof sizes and the GPU speed-up. Use ≥ 5 repetitions per configuration.

T-E2-02 Full RQ2 latency benchmark fills: `Sec. 6.5 slot`
hardware: A100 · est. wall-clock: about 30–40 h (30 seeds × 60 min, plus proving backlog)

- **Warning.** `make e2-strict` still runs the *legacy mock* pipeline (`e2-bench`, with a mock L1 delay). Do **not** use it.
- Use the real runner:
  ```
  cargo run -release -manifest-path crates/sp1-e2e/Cargo.toml -p sp1-e2e -features cuda \
    -bin e2_real_benchmark - -lambda-events-per-min 480 -duration-min 60 -seeds 30 \
    -prover cuda -out /datastore/uitchain/e2/raw.jsonl -run-id <unique>
  python3 scripts/e2_real_analyze.py ...
  ```
- Report median, IQR, P95 and P99, plus queue saturation. If the prover cannot keep up at 480 events/min, report saturation honestly. Then add the highest sustainable λ as an extra row; do not shorten the run.

T-E3-02 RQ3: 4 modes × 30 seeds fills: `Table 5 (all cells)`
hardware: A100 · est. wall-clock: about 120 runs × ≤2 h; use a Slurm array, one topology per job

- Run each mode and seed, with mode order rotated per seed:
  ```
  make e3-run E3_MODE=<fabric-only|hash-on-chain|vecro-adapted|proposed> E3_SEED=<0..29>
  E3_RAW_ROOT=/datastore/uitchain/e3/raw
  ```

- Analyse:
  ```
  make e3-analyze E3_RAW_ROOT=/datastore/uitchain/e3/raw
  ```
- **Acceptance.** `results/e3/comparison.json` contains the formal Wilcoxon and Bonferroni results; the analyser refuses partial data.
- Fill every Table 5 cell with median [IQR]. Add one sentence giving the p-values and Cliff's δ for the proposed mode against the VeCroToken-adapted mode.

T-COST-01 Gas → fiat conversion — fills: `Sec. 6.7 slot`
hardware: dev laptop · est. wall-clock: 0.5 day

- Write a script `scripts/gas_to_usd.py` that takes the gas from the receipts, a stated gas price (gwei) and an ETH/USD rate, with a date and source cited.
- Report a sensitivity range: low, median and high gas price over a stated window.
- Never present Anvil gas as a mainnet fee without these stated assumptions.

T-E4-01 RQ4 screening (80 runs) — fills: `Sec. 6.7 slot`
hardware: A100 · est. wall-clock: about 1–2 days

- Run each screening cell, then merge:
  ```
  make e4-run E4_PHASE=screening E4_PLAN_INDEX=<0..79> E4_RUN_ROOT=/datastore/uitchain/e4 \
    E3_GATE_MANIFEST=results/e3/gate_manifest.json E4_PREFLIGHT=results/e4/preflight.json
  make e4-merge E4_PHASE=screening E4_RUN_ROOT=/datastore/uitchain/e4
  ```

T-E4-02 RQ4 confirmation (270 runs) and Figure 2 — fills: `Fig. 2 (fig:rq4)`
hardware: A100 · est. wall-clock: about 3–5 days

- Run as `T-E4-01` with `E4_PHASE=confirmation` and `E4_PLAN_INDEX=0..269`, then:
  ```
  make e4-analyze E4_RAW=...  E4_MANIFEST=...
  ```
- Draw Figure 2 as a standalone PGFPlots figure with the `/draw` skill:
  – panel (a): throughput and completion versus shipment size, one line per epoch length;
  – panel (b): gas and calldata per shipment.
- Show median and IQR, hatch censored or saturated cells, and size the figure to `\textwidth`×34 mm.

T-SEP-01 Optional public-testnet run — fills: `Sec. 7 slot`
hardware: any · est. wall-clock: 0.5 day

- Deploy `EpochAggregateAnchor` on Sepolia. Anchor 5 epochs and record the inclusion delay.
- If this is skipped, delete the slot and keep "Anvil only" in the limitations.

4 Manuscript finalisation

T-ABS-01 Headline numbers in the abstract and conclusion — fills: `Abstract, Sec. 8`
hardware: – · est. wall-clock: 1 h

- After `T-E2-02` and `T-E3-02`, write one sentence each, for example: "Over 30 seeds at 480 events/min, median audit latency was X s (P95 Y s), and per-shipment settlement gas fell Z-fold relative to the VeCroToken-adapted mode ($p < 0.001$)."

T-DOC-01 Authors' ORCIDs, funding and acknowledgement — fills: `Credits block`
hardware: – · est. wall-clock: 1 h

- Add `\orcidID{...}` for each author in `main.tex`.
- Add the funding source (grant number, e.g. UIT/VNU-HCM) to `\ackname`.

T-FIN-01 Final gate before EasyChair — fills: -
hardware: – · est. wall-clock: 2 h

- Set `\finaltrue`; the file must compile with no pending slot.
- Check page count ≤ 15 (`pdfinfo`) and that there are no overfull boxes.
- Check that every new number is listed in `evidence_ledger.md`.
- Re-run `/ready`.
- Tag the repository version cited in the paper (`git tag aciids2027-v1`) and replace the footnote URL with the tag.

5 Schedule (backward from 31 October 2026)

Dates	Tasks	Owner (suggested)
5–9 Oct	T-SEC-01, T-SEC-03, T-SEC-02 (ABI freeze on 9 Oct); T-E2-01	Van-Thinh Ngo
8–10 Oct	T-E1-01, T-E1-02 (Path B, independent of the ABI)	Duy-Nhan Tran
10–12 Oct	T-E3-01 gate; T-E1-03 A/B preflight	Huu-Tri Phan
12–16 Oct	T-E2-02 (30 seeds); T-DM-01; T-SD-01	Ngo / Tran
13–23 Oct	T-E3-02 (120 runs, Slurm array); T-COST-01	Huu-Tri Phan
20–28 Oct	T-E4-01, then T-E4-02 and Fig. 2	Duy-Nhan Tran
28–29 Oct	Fill all slots, T-ABS-01, T-DOC-01; internal review	Tuan-Dung Tran
30 Oct	T-FIN-01, submit to EasyChair	Tuan-Dung Tran

Fallback if compute runs late.

- If E4 confirmation is incomplete by 28 Oct, report the 80-run screening grid only, with the single-seed caveat stated in the caption.
- If E3 is incomplete, the paper must *not* claim the comparison. Delete Table 5, keep the analytic mG_v/G_a argument with the measured G_v and G_a from `T-E3-01`, and reframe RQ3 as a cost model.
- Never fill a slot with pilot, mock or accelerated data.